

Credential Handling for the TCA Integration Account

AthleticOS · prepared for the Blackbaud access discussion · 14 September 2026

This covers the questions Omar raised: how the credential is stored, how it gets rotated, how TCA kills it, what gets logged, and what happens to all of it when the pilot ends. Written against what the code actually does today, with the gaps marked rather than glossed.

BUILT in the code now **BEFORE FIRST CALL** committed, not yet built **OPEN** needs a decision with TCA

One correction to carry into the conversation

The access-model one-pager TCA already has says *“Every call is logged with timestamp, endpoint and response code, available to TCA on request.”* That logging is not built. No call has ever been made, so nothing has gone unrecorded, but the statement only becomes true if the audit trail exists before the integration first runs. It is listed below as **BEFORE FIRST CALL** and should be described that way if it comes up.

What TCA is being asked for

A service account with read-only permissions, used by AthleticOS to read three things on an ongoing basis:

- The school calendar
- Athletics teams
- Team schedules

No rosters, so no student records in this phase. No write access of any kind. Blackbaud security groups: `SKY API Basic`, `SKY API Data Sync`, `Sky Reporting`.

Why read-only has to live on the account **BUILT**

SKY API has no per-application permission scopes. Blackbaud's OIDC discovery document omits `scopes_supported` entirely, and an issued token inherits whatever the authorising account is allowed to do. There is no request AthleticOS can make that narrows its own access below the account's permissions.

So read-only on the service account is not a courtesy on our side. It is the only enforcement point that exists. If the account cannot write, the integration cannot write, regardless of what our code does or what anyone promises.

What is stored, and where BUILT

AthleticOS never stores a TCA password. The OAuth authorisation-code flow is run once against the service account, and what persists afterwards is a token pair.

FIELD	WHAT IT IS	AT REST
<code>access_token</code>	Short-lived bearer token	Encrypted
<code>refresh_token</code>	Used to obtain new access tokens	Encrypted
<code>expires_at</code>	Access token expiry	Plain timestamp
<code>environment_id</code>	Confirms which tenant a token belongs to	Plain

One row per school in `blackbaud_connections` , on the PostgreSQL instance behind AthleticOS at Render (US East, SOC 2 Type II certified provider). Deleting the school deletes the row.

Encryption at rest BUILT

Both tokens are encrypted at the application layer before they reach the database, so they are unreadable to anyone holding only a database dump or a disk snapshot. This sits above Render's own at-rest disk encryption, not instead of it.

- **Cipher:** AES-256-GCM, 12-byte random IV per value, authentication tag retained and verified on read.
- **Key derivation:** HKDF-SHA256 (RFC 5869) with a domain-separation string of `athleticos:blackbaud:token-v1` .
- **Stored format:** `v1:<iv>:<authTag>:<ciphertext>` , each part base64url. The version prefix allows a future format without ambiguity.
- Tampering with a stored value fails the GCM auth tag and raises rather than returning wrong plaintext.

Worth stating plainly if asked: the encryption key is derived from the application's existing signing secret rather than from a separate KMS. Domain separation via the HKDF info string means compromising one does not directly yield the other, but both live in the same environment. Moving this to a dedicated managed key is a known improvement and we would take a TCA requirement here as a reason to do it.

Who can reach the credential OPEN

- **The application** reads and decrypts it in memory to sign requests. It is never written to logs, never returned by any API response, and never rendered in the UI.

- **Burke Autrey** holds the Render account, so in practice he can reach the environment and therefore the key material. That is the honest answer: the credential is protected from outside access, not from the operator.
- **Truman Blocker** has no infrastructure access.

If TCA wants a second pair of eyes on that, the practical answer is that TCA holds the account itself and can revoke it unilaterally at any moment, which is a stronger control than any promise about operator restraint.

Rotation BEFORE FIRST CALL

- **Access tokens** rotate on their own, expiring in about an hour and being re-obtained via the refresh token. Nothing manual.
- **Refresh tokens** are replaced whenever Blackbaud issues a new one, and the stored value is overwritten with the re-encrypted replacement.
- **The service account password** is TCA's to rotate on TCA's schedule. A rotation invalidates our tokens, we re-run the one-time authorisation, and the integration resumes. We would like notice, but nothing breaks permanently if we do not get it.
- **Scheduled re-authorisation:** we propose re-running the authorisation at the 90-day pilot boundary whether or not anything has expired, so a stale grant cannot quietly outlive the pilot.

How TCA kills it BUILT

Two independent paths, and the first does not depend on AthleticOS doing anything:

- **TCA side, unilateral:** revoke the application or disable the service account in the Blackbaud admin portal. Every token dies immediately. No code change, no notice to us, no cooperation required.
- **AthleticOS side:** a disconnect endpoint deletes the stored connection row. It is restricted to school administrators and athletic directors, so a coach account cannot trigger it.

The first is the one that matters. TCA should never be in a position where switching us off requires us to act.

Audit trail BEFORE FIRST CALL

The commitment is that every Blackbaud call is recorded with timestamp, endpoint, and response code, and that the record is available to TCA on request. This is what the one-pager promised and it is not yet implemented.

What it needs is a write to an append-only table on every outbound call. Deliberately recorded: time, HTTP method, endpoint path, response status, and the acting user where one initiated the request. Deliberately not recorded: response bodies, so the audit trail does not itself become a second copy of student data.

Our position should be that this is built and demonstrable before the first production call, not that it exists today.

End of the pilot [OPEN](#)

- TCA disables the service account. That alone ends all access.
- We delete the stored connection row and the tokens with it.
- Any TCA data held in AthleticOS is exported to TCA in CSV or JSON on request, then deleted within 30 days, per the signed pilot agreement.
- The audit trail is the one thing worth keeping past deletion, and we would keep it only with TCA's agreement, since it is a record of our own access rather than school data.

The signed agreement puts the term at 90 days from access grant, renewable only by mutual agreement.

The MFA question [OPEN](#)

Raised early because it is the item most likely to stall this quietly. If TCA policy requires multi-factor on every account, an unattended integration cannot satisfy an interactive second factor on token refresh. The options, in the order we would prefer them:

1. A documented policy exception for one non-interactive service account, compensated by it being read-only, single-purpose, audited, and revocable in one click.
2. A Blackbaud-native service or application credential, if Education Management offers one that avoids an interactive login.
3. Interactive authorisation performed by a TCA administrator, with re-authorisation needed whenever the grant lapses. This works but puts a recurring task on Omar, which is worth avoiding.

This is TCA's policy call, not ours. We only need to know which path before building against it.

Summary

QUESTION	ANSWER	STATUS
Where does the credential live?	Encrypted token pair, one row, Postgres at Render US East	BUILT
How is it protected?	AES-256-GCM, HKDF-derived key, app layer, above disk encryption	BUILT
What stops writes?	The account's own permissions; SKY has no per-app scopes	BUILT
How does TCA revoke?	Blackbaud admin portal, unilateral, immediate	BUILT
How does it rotate?	Tokens automatically; account password on TCA's schedule	BUILT
Re-authorise at pilot boundary	Proposed at 90 days	BEFORE FIRST CALL
What is logged?	Timestamp, endpoint, status, acting user; no response bodies	BEFORE FIRST CALL
Who can reach it internally?	The app; and Burke, as operator of the hosting account	OPEN
MFA on the service account	Needs a TCA policy decision	OPEN
End of pilot	Account disabled, tokens deleted, data returned then deleted in 30 days	OPEN

AthleticOS — Burke Autrey and Truman Blocker. Prepared 14 September 2026 for Trinity Christian Academy. Technical statements verified against the AthleticOS source at commit `fa85626` ; items marked “Before first call” are commitments, not current behaviour.